

# **SE411 Course Project**

## **Vehicle Asset Management System**

Supervisor: Dr. Hazleen Binti Aris

Prepared By:

Norah Alarifi - 223410672

Ghala Aljarallah - 222410294

Jana Altalhi - 221410589

## Project Overview

The Company Vehicle Asset Management System is a full-stack web application designed to manage company transportation vehicle assets. It provides a complete Create, Read, Update, and Delete (CRUD) interface for vehicle records, built on a React + TypeScript frontend backed by a Spring Boot REST API with an H2 in-memory database. The system supports five distinct vehicle asset classes: Car, Van, Truck, Bus, and Motorcycle. Each with its own type-specific attributes in addition to shared common fields.

## Requirements

The core functional requirements implemented in the system are:

- **Full-Stack Architecture:** A React + TypeScript Single Page Application communicates with a Spring Boot backend via a REST API.
- **Multi-Type Asset Management:** The system manages five vehicle asset classes, each with type-specific fields rendered dynamically in the form.
- **Data Initialization:** The backend seeds the database on startup with one sample record per vehicle type using `schema.sql` and `data.sql`.
- **Add:** A dedicated form allows users to register new vehicle assets, with type-specific fields that update based on the selected vehicle type.
- **Read:** An interactive list view displays all vehicle records with real-time search and automatic sorting by last updated.
- **Update:** Existing records can be edited through the same form, pre-filled with the current values including type-specific fields.
- **Delete:** Records can be deleted from the list view with a confirmation prompt.
- **Navigation:** A persistent navigation menu provides access to all system features: Dashboard, Vehicle List, Add Vehicle, and About.
- **About Page:** Displays team member information and a description of the project scope.

## Data model

The system captures and manages the following data points for every vehicle asset. All types share the common fields defined in the `VehicleProps` interface:

- **Unique Identifier (id):** A server-generated unique identifier prefixed with `veh_`.
- **Asset Tag:** A unique business identifier used for tracking (e.g., TRN-001).
- **Vehicle Type:** One of Car, Van, Truck, Bus, or Motorcycle — determines which type-specific fields apply.
- **Make, Model, and Year:** Core vehicle specifications.
- **Plate Number:** Official vehicle registration detail.

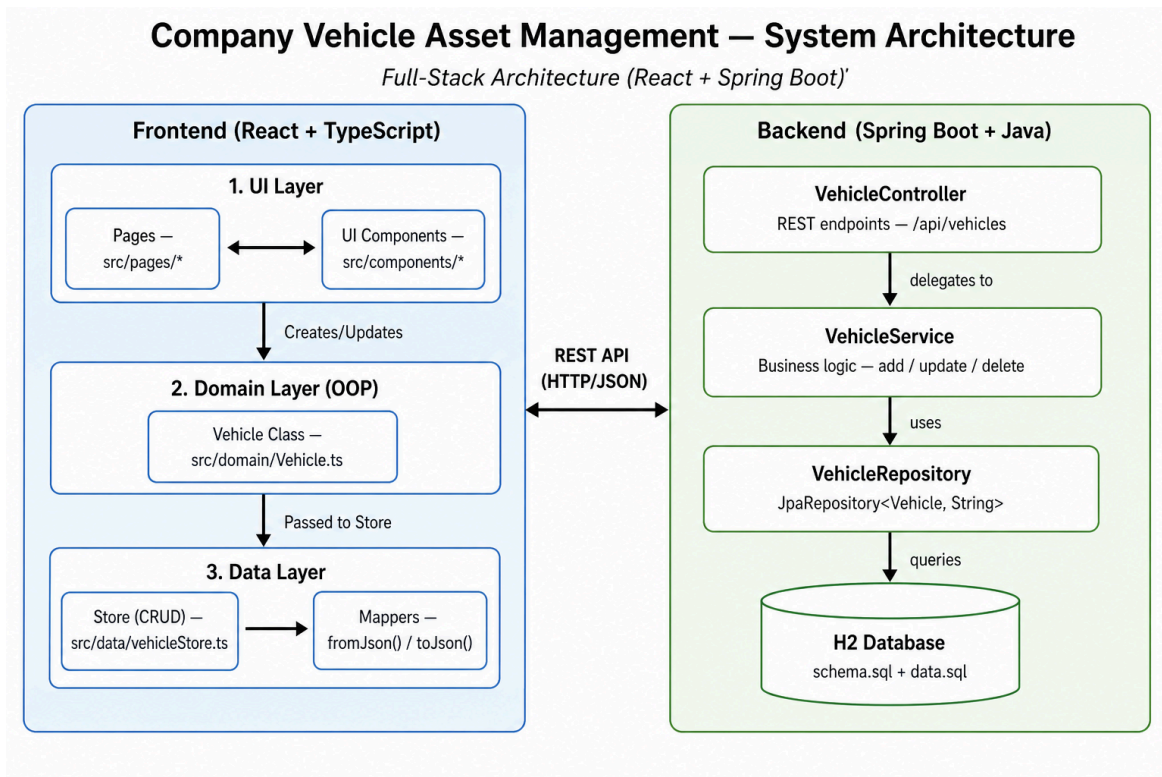
- **Status:** Current operational condition — Active, In Maintenance, or Retired.
- **Assigned Department:** The department responsible for the asset.
- **Notes:** Optional free-text context field.
- **Audit Timestamps:** **createdAtIso** and **updatedAtIso** are set automatically on creation and update.

Each vehicle type additionally carries its own specific fields:

- **Car:** Number of doors, fuel type (Petrol, Diesel, Hybrid, Electric).
- **Van:** Cargo volume in m<sup>3</sup>, sliding door indicator.
- **Truck:** Payload capacity in kg, refrigeration unit indicator.
- **Bus:** Seating capacity, articulated indicator.
- **Motorcycle:** Engine displacement in CC, sidecar indicator.

## System Architecture

The system follows a full-stack layered architecture that separates concerns across a React frontend and a Spring Boot backend, connected through a REST API.



**[FIGURE 1]** A block diagram depicting the full-stack layered architecture, showing the Frontend (UI Layer, Domain Layer, Data Layer) communicating via REST API with the Backend (Controller, Service, Repository, and H2 Database).

## Frontend — UI Layer (React)

React components and React Router handle all user interaction and rendering.

- **Routing (`App.tsx`):** Maps application URLs to page components (e.g., `/` to `HomePage`, `/vehicles` to `VehicleListPage`).
- **Shell (`AppShell.tsx`):** Provides the persistent layout including the header, navigation menu, and footer.
- **Page Components (`src/pages/`):** Primary views for each route. They handle logic, communicate with the data layer, and render UI components.
- **UI Components (`src/components/`):** Reusable elements including buttons, cards, badges, and `VehicleForm.tsx` which dynamically renders type-specific fields based on the selected vehicle type.

## Frontend — Domain Layer (OOP)

- **`Vehicle.ts`:** The core frontend model. Defines `VehicleProps` — the shared data structure for all vehicle types including type-specific optional fields. The `create()` static method instantiates new valid vehicle objects, trims input, and sets timestamps. The `update()` method enforces which fields may be modified and automatically sets `updatedAtIso`.

## Frontend — Data Layer

- **`vehicleStore.ts`:** Exposes clean async functions (`listVehicles`, `addVehicle`, `updateVehicle`, `deleteVehicle`) used by page components. Contains `fromJson()` and `toJson()` mappers that translate between backend JSON field names and frontend `VehicleProps` field names.
- **`VehicleRepository.ts`:** Maintains browser-side seed data consistency for offline fallback.

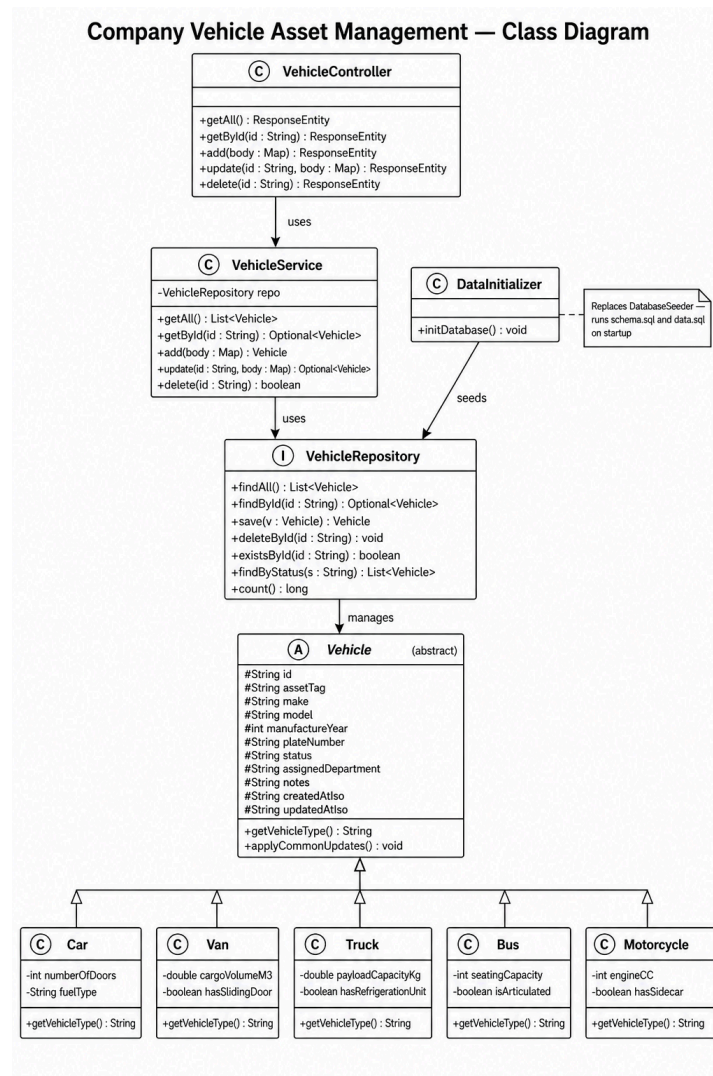
## Backend — Spring Boot REST API

- **`VehicleController`:** Exposes five REST endpoints (`GET /api/vehicles`, `GET /api/vehicles/{id}`, `POST /api/vehicles`, `PUT /api/vehicles/{id}`, `DELETE /api/vehicles/{id}`). Accepts JSON request bodies with a `vehicleType` discriminator field.
- **`VehicleService`:** Contains all business logic. The `add()` method reads the `vehicleType` field and instantiates the correct subclass. The `update()` method uses Java pattern-matching `instanceof` to apply type-specific field updates.

- **VehicleRepository**: A single Spring Data JPA interface extending `JpaRepository<Vehicle, String>`. Handles all five vehicle types through the base class — no separate repository per type is needed because Hibernate resolves the correct subclass at runtime through the JOINED inheritance strategy.

## Backend — Model Layer (Java OOP)

The backend uses JPA JOINED inheritance. **Vehicle** is an abstract base class mapped to the `vehicles` table. Each subclass has its own table joined by a foreign key. The `vehicle_type` discriminator column identifies the subclass for each row.

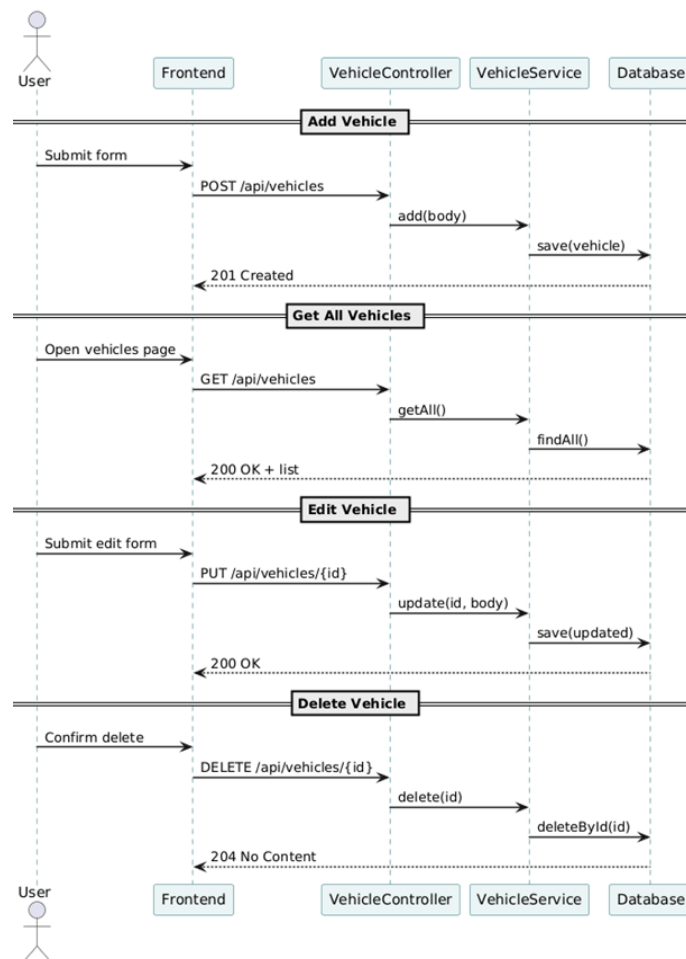


**[FIGURE 2]** UML class diagram showing the *Vehicle* abstract base class with five subclasses (*Car*, *Van*, *Truck*, *Bus*, *Motorcycle*), *VehicleController*, *VehicleService*, and *VehicleRepository*.

- **Car:** numberOfDoors, fuelType
- **Van:** cargoVolumeM3, hasSlidingDoor
- **Truck:** payloadCapacityKg, hasRefrigerationUnit
- **Bus:** seatingCapacity, isArticulated
- **Motorcycle:** engineCC, hasSidecar

## Backend — Database

The H2 in-memory database is initialised on every startup using `schema.sql` (which creates all six tables) and `data.sql` (which seeds one sample record per vehicle type). Hibernate's `ddl-auto` is set to `none` so the SQL scripts have full control over the schema.



**[FIGURE 3]** Sequence diagram showing the request flow for Add, Get, Edit, and Delete vehicle operations between the Frontend, VehicleController, VehicleService, and Database.

## User Interface Framework and Tools

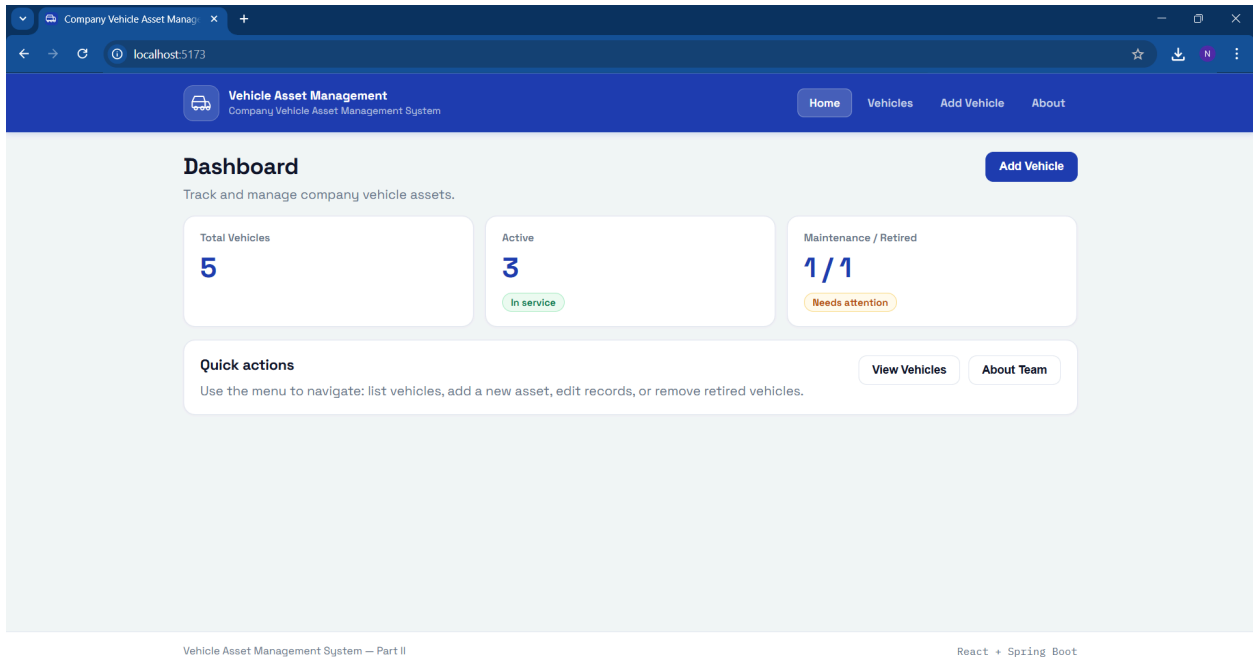
- **React.js:** The primary UI library used to create components and handle application state.
- **TypeScript:** Adds strict static typing to JavaScript, uncovering errors early in development and improving code maintainability and readability.
- **Vite:** The build tool and development server, chosen for its fast startup and hot-reload times.
- **React Router:** Manages client-side navigation and routing between pages within the Single Page Application.
- **CSS Variables:** Used throughout `src/styles/app.css` to maintain a consistent color scheme, spacing, and typography across the entire UI.

## Backend Framework and Tools

- **Spring Boot 3.2:** Provides the REST API, dependency injection, and application lifecycle management.
- **Java 17:** The backend language, using modern features such as pattern-matching `instanceof` for clean subclass handling in `VehicleService`.
- **Spring Data JPA / Hibernate:** Manages the ORM layer, translating Java entity classes to relational database tables using the JOINED inheritance strategy.
- **H2 Database:** An in-memory relational database used for development. The schema and seed data are managed entirely through `schema.sql` and `data.sql`, loaded automatically on startup.

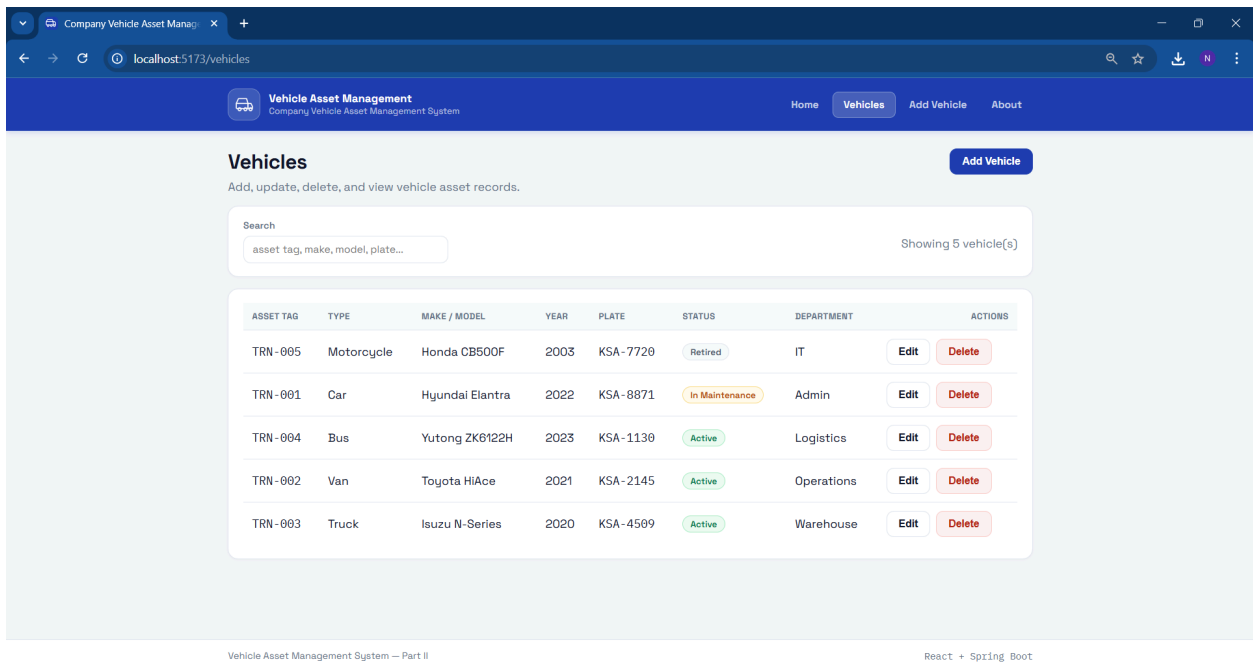
## Implementation

### System Dashboard



**[FIGURE 4 — Dashboard]** *The system dashboard displaying the total number of vehicle assets (5), active vehicles (3), and vehicles in maintenance or retired (1/1), with quick navigation actions.*

## Vehicle List and Management



**[FIGURE 5 — Vehicle List page]** *The vehicle list page displaying all five registered assets in a table, showing asset tag, vehicle type, make and model, year, plate number, status badge, assigned department, and Edit/Delete action buttons.*

## Edit Vehicles

The screenshot shows a web browser window with the URL `localhost:5173/vehicles/veh_005/edit`. The page title is "Edit Vehicle" and the subtitle is "Update asset: TRN-005". The form is pre-filled with the following data:

| Field               | Value                         |
|---------------------|-------------------------------|
| Asset Tag           | TRN-005                       |
| Vehicle Type        | Motorcycle                    |
| Make                | Honda                         |
| Model               | CBS00F                        |
| Year                | 2003                          |
| Plate Number        | KSA-7720                      |
| Status              | Retired                       |
| Assigned Department | IT                            |
| Engine CC           | 471                           |
| Has Sidecar         | No                            |
| Notes               | Used for rapid IT dispatches. |

At the bottom right of the form, there are two buttons: "Cancel" and "Save changes".

**[FIGURE 6 — Edit Vehicle Form]** *The Edit Vehicle form pre-filled with existing asset data for a Motorcycle (TRN-005), showing the dynamically rendered Motorcycle-specific fields (Engine CC and Has Sidecar).*

- **Dynamic Type Fields:** When a vehicle type is selected, the form automatically renders the fields specific to that type. For example, selecting Truck shows Payload Capacity (kg) and Refrigeration Unit, while selecting Bus shows Seating Capacity and Articulated.
- **Form Validation:** Required fields are validated on submission. The year must fall within a valid range. Submission is blocked until all errors are resolved.
- **Real-Time Error Display:** Validation errors are shown in a summary alert box above the action buttons.